

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion-HOWTO](#)



[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[WinCVS](#) › [TortoiseCVS](#) › [DocbookSgml/CVS_Tutorial-KLDP](#) › [DocbookSgml/CVS-KLDP](#) › [cvs팁](#) ›

[Subversion-HOWTO](#)

Subversion 사용 HOWTO

이재홍 <http://www.pyrasis.com> ~2005.7.31 버전 1.2.1

Login:

Password:

[Join](#)

Good fortune in
as a better posi

CVS의 단점들을 개선한 버전 관리 시스템인 Subversion을 이용하여 프로그램의 소스 코드를 관리하는 방법과 유닉스, 리눅스 및 Windows에서 Subversion을 설치해보고 사용하는 방법을 설명합니다.

[HelpOnEditing/Su](#)
[giljae](#)
[romance](#)
[InterruptAndExce](#)

Contents▲

[1 소프트웨어 버전 관리의 이해](#)

- [1.1 버전 관리 시스템의 필요성](#)
- [1.2 버전 관리 시스템의 종류](#)
- [1.3 버전 관리 시스템의 용어들](#)
- [1.4 저장소의 디렉토리 배치](#)

[2 Subversion](#)

- [2.1 CVS와 비교한 Subversion의 장점들](#)
- [2.2 설치 준비 작업](#)
- [2.3 사용 할 각각의 파일들 구하기](#)

[3 설치하기](#)

- [3.1 OpenSSL 컴파일과 설치](#)
- [3.2 Berkeley DB 컴파일과 설치](#)
- [3.3 Apache 컴파일과 설치](#)
- [3.4 Subversion 컴파일과 설치](#)

[4 세부 설정](#)

- [4.1 저장소 만들기](#)
 - [4.1.1 공동 작업을 위한 저장소 그룹 설정](#)
- [4.2 Apache 설정](#)
 - [4.2.1 Apache에서 ID로 사용자 인증](#)
- [4.3 svnserve를 사용한 서버](#)
 - [4.3.1 svnserve에서 ID로 사용자 인증](#)
- [4.4 SSH + svnserve 서버](#)

[5 실제로 사용하기](#)

- [5.1 에디터 설정](#)
- [5.2 기본 디렉토리 만들기](#)
- [5.3 Import](#)
- [5.4 Checkout](#)
- [5.5 Update](#)
- [5.6 Commit](#)
- [5.7 Log](#)
- [5.8 Diff](#)
- [5.9 Blame](#)
- [5.10 lock](#)
- [5.11 Add](#)
- [5.12 Export](#)
- [5.13 Branch와 Tag](#)
 - [5.13.1 Branch](#)
 - [5.13.1.1 Merge](#)
 - [5.13.2 Tag](#)
- [5.14 Revert](#)
- [5.15 백업 및 복구](#)
 - [5.15.1 Dump](#)
 - [5.15.2 Load](#)

[6 Microsoft Windows에서 사용하기](#)

- [6.1 설치 파일 구하기](#)
- [6.2 설치](#)
- [6.3 사용하기](#)

[7 운영체제별 전용 패키지](#)

[8 GUI 클라이언트 프로그램](#)



kss@kld

[8.1 TortoiseSVN](#)

[8.2 Ankhsvn](#)

[8.3 RapidSVN](#)

[9 웹 인터페이스](#)

[9.1 ViewCVS](#)

[9.2 WebSVN](#)

[10 쓰다가 발생하는 사소한 문제](#)

[10.1 svn: Entry 'file name' has unexpectedly changed special status](#)

1 소프트웨어 버전 관리의 이해 ¶

[[edit](#)]

Subversion은 소프트웨어 버전 관리 시스템입니다. 이전에 CVS같은 역사가 깊은 소프트웨어 버전 관리 시스템을 사용해 본 경험이 있다면 쉽게 진행 할 수 있을 것입니다. 이 장에서는 소프트웨어 버전 관리 시스템을 처음 접하는 분들을 위해 자주 나오는 용어들과 개념을 정리 하였습니다.

1.1 버전 관리 시스템의 필요성 ¶

[[edit](#)]

파연 소프트웨어를 만드는데 버전관리가 왜 필요할까요? 버전 관리가 필요하게 된 이유는 공동 작업 때문입니다. 한사람이 큰 프로젝트를 진행 하는 것이 아니기 때문에 버전 관리 시스템이 필요 하게 되었습니다.

버전 관리 프로그램을 사용하게 되면 다음과 같은 장점이 있습니다.

- 개발 버전과 릴리즈 버전을 섞이지 않고 쉽게 관리 할 수 있습니다.
- 소스를 잘 못 수정 했더라도 기록이 남고 되돌리기가 쉽습니다.(많은 파일의 경우 유용)
- 수정, 추가, 삭제 등의 기록이 모두 남고 변경 사항을 추적하기 쉽습니다.
- 개발자들이 따로 따로 백업을 하지 않아도 됩니다.

가장 유용한 장점은 아무래도 잘못 수정한 소스를 쉽게 되돌릴 수 있다는 것입니다. 프로젝트가 커지다보면 소스가 꼬이게 되고 골치 아픈 상황이 한두 번 발생하는 것이 아니죠. 그리고 변경 사항이 모두 기록되고 무엇을 변경했는지 쉽게 볼 수 있습니다. 옆 사람이 수정한 것을 쉽게 볼 수 있습니다. 그리고 가장 큰 문제를 일으키는 백업을 하지 않아 소스를 분실하는 최악의 상황도 쉽게 해결 됩니다.

장점을 나열하자면 더 많습니다만 단점을 이야기 하자면 아무래도 프로그램 개발자들이 소프트웨어 버전 관리 시스템의 개념을 제대로 이해하고 기능들을 잘 사용하여 효율적인 버전 관리가 되려면 또다시 새로운 것을 배워야 한다는 것 때문에 쉽게 접근하지 않으려는 경향이 있습니다. 특히나 우리나라 프로그램 개발자들의 경우 윈도우 공유 폴더를 이용해서 개발을 하는 경우가 많습니다. 이렇게 되면 누가 무엇을 수정했는지 알 수도 없고 남이 해 놓은 소스 위에 잘못된 소스를 덮어쓰는 경우도 많이 발생하고 있습니다.

한사람이 개인적으로 진행 하는 프로젝트가 있더라도 버전 관리 프로그램을 사용하는 것이 매우 편리합니다. 앞서 말한 장점들은 여러 명이 개발 하는 것과 한사람이 개발 하는 것에 모두 해당되는 것들입니다.

소프트웨어 버전 관리 시스템을 이용하는 프로젝트들은 아주 많습니다. 대부분 Open Source로 된 프로젝트들은 CVS를 이용하여 버전 관리를 합니다. 대표적으로 *BSD, [OpenOffice](#), Mozilla, [XFree86](#), Apache와 [SourceForge.net](#)의 모든 프로젝트들 입니다. 비단 Open Source 뿐만이 아닌 상업 프로그램을 만드는 회사들에서도 소프트웨어 버전 관리 시스템은 필수적입니다.

1.2 버전 관리 시스템의 종류 ¶

[[edit](#)]

현재 나와 있는 소프트웨어 버전 관리 시스템은 여러 종류가 있습니다. 각각 장단점이 있습니다.

- CVS (Concurrent Version System) : 가장 널리 사용되며 역사가 깊은 버전 관리 시스템입니다. <http://www.cvshome.org>
- Subversion : CVS의 단점을 개선하고 CVS를 대체할 목적으로 개발 되었습니다. 이 문서에서 설명할 버전 관리 시스템입니다. <http://subversion.tigris.org>
- Visual Sourcesafe : Microsoft에서 만든 버전 관리 시스템입니다. CVS와는 버전 관리 관점에서 조금의 차이점이 있습니다. 윈도우 기반 소프트웨어의 버전 관리를 할 때 자주 사용됩니다. <http://msdn.microsoft.com/ssafe/>
- Clear Case : Rational이라는 회사에서 만든 버전 관리 시스템입니다. 지금은 IBM에 합병되었습니다. 상용 소프트웨어입니다. <http://www-306.ibm.com/software/rational/>
- BitKeeper : 리눅스 커널이 BitKeeper를 이용해서 개발 하고 있습니다. 상용 소프트웨어입니다. <http://www.bitkeeper.com>

1.3 버전 관리 시스템의 용어들 ¶

[[edit](#)]

소프트웨어 버전 관리 시스템에서 사용되는 용어들을 알아보도록 하겠습니다.

저장소 : 리포지토리(Repository)라고도 하며 모든 프로젝트의 프로그램 소스들은 이 저장소 안에 저장됩니다. 그리고 소스뿐만이 아니라 소스의 변경 사항도 모두 저장됩니다. 네트워크를 통해서 여러 사람이 접근 할 수 있습니다. 버전 관리 시스템 마다 각각 다른 파일 시스템을 가지고 있으며 Subversion은 Berkeley DB를 사용합니다. 한 프로젝트 마다 하나의 저장소가 필요합니다.

체크아웃 : 저장소에서 소스를 받아오는 것입니다. 체크아웃을 한 소스를 보면 프로그램 소스가 아닌 다른 디렉토리와 파일들이 섞여 있는 것을 볼 수 있습니다. 이 디렉토리와 파일들은 버전 관리를 위한 파일들입니다. 임의로 지우거나 변경하면 저장소와 연결이 되지 않습니다. 체크아웃에도 권한을 줄 수 있습니다. 오픈 소스 프로젝트들에서는 대부분 익명 체크아웃을 허용하고 있습니다.

커밋(Commit) : 체크아웃 한 소스를 수정, 파일 추가, 삭제 등을 한 뒤 저장소에 저장하여 갱신 하는 것입니다. 커밋을 하면 CVS의 경우 수정한 파일의 리비전이 증가하고 Subversion의 경우 전체 리비전이 1 증가하게 됩니다.

업데이트(Update) : 체크아웃을 해서 소스를 가져 왔더라도 다른 사람이 커밋을 하여 소스가 달라졌을 것입니다. 이럴 경우 업데이트를 하여 저장소에 있는 최신 버전의 소스를 가져옵니다. 물론 바뀐 부분만 가져옵니다.

리비전(Revision) : 소스 파일등을 수정하여 커밋하게 되면 일정한 규칙에 의해 숫자가 증가 합니다. 저장소에 저장된 각각의 파일 버전이라 할 수 있습니다. Subversion의 경우 파일별로 리비전이 매겨지지 않고 한번 커밋한 것으로 전체 리비전이 매겨 집니다. 리비전을 보고 프로젝트 진행 상황을 알 수 있습니다.

임포트(Import) : 아무것도 들어있지 않은 저장소에 맨 처음 소스를 넣는 작업입니다.

익스포트(Export) : 체크아웃과는 달리 버전 관리 파일들을 뺀 순수한 소스 파일을 받아올 수 있습니다. 소스를 압축하여 릴리즈 할 때 사용합니다.

1.4 저장소의 디렉토리 배치 ¶

[[edit](#)]

저장소에 바로 소스를 넣어 프로젝트를 진행 할 수 있습니다. 그렇지만 버전 관리 시스템에서 권장하는 디렉토리 배치 방법이 있습니다.

-- <http://svn.samplerepository.org/svn/sample>

```
+---+---+- branches
|   +---+- dav-mirror
|   |   |--- src
|   |   |--- doc
|   |   +--- Makefile
|   |
|   +--- svn-push
|   +--- svnserve-thread-pools
|
+---+---+- tags
|   +--- 0.10
|   +---+ 0.10.1
|   |   |--- src
|   |   |--- doc
|   |   +--- Makefile
|   |
|   +--- 0.20
|   +--- 0.30
|   +--- 0.50
|   +--- 1.01
|
+---+---+- trunk
|   |--- src
|   |--- doc
|   +--- Makefile
```

위에 보이는 구조는 보통 자주 사용되는 디렉토리 구조입니다. 저장소 디렉토리 아래 branches, tags, trunk 라는 3개의 디렉토리가 있습니다. 이 디렉토리들은 각각의 용도가 있습니다. CVS는 branch와 tag를 위한 명령이 따로 존재 하지만, Subversion의 경우 명령이 있긴 하지만 단순한 디렉토리 복사와 같은 효과를 냅니다.

trunk : 단어 자체의 뜻은 본체 부분, 나무줄기, 몸통 등 입니다. 프로젝트에서 가장 중심이 되는 디렉토리입니다. 모든 프로그램 개발 작업은 trunk 디렉토리에서 이루어 집니다. 그래서 위의 구조에서 trunk 디렉토리 아래에는 바로 소스들의 파일과 디렉토리가 들어가게 됩니다.

branches : 나무줄기(trunk)에서 뻗어져 나온 나무 가지를 뜻합니다. trunk 디렉토리에서 프로그램을 개발하다 보면 큰 프로젝트에서 또 다른 작은 분류로 뺄 때 따로 개발해야 할 경우가 생깁니다. 프로젝트안의 작은 프로젝트라고 생각하면 됩니다. branches 디렉토리 안에 또 다른 디렉토리를 두어 그 안에서 개발하게 됩니다.

tags : tag는 꼬리표라는 뜻을 가지고 있습니다. 이 디렉토리는 프로그램을 개발하면서 정기적으로 릴리즈를 할 때 0.1, 0.2, 1.0 하는 식으로 버전을 붙여 발표하게 되는데 그때그때 발표한 소스를 따로 저장하는 공간입니다. 위에서 보면 tags 디렉토리 아래에는 버전명으로 디렉토리가 만들어져 있습니다.

2 Subversion ¶ [\[edit\]](#)

Subversion은 CVS를 대체하기 위해 개발되고 있습니다. Subversion은 소스 코드는 물론 바이너리 파일 등의 여러가지 형식의 파일을 관리 할 수 있습니다.

2.1 CVS와 비교한 Subversion의 장점들 ¶ [\[edit\]](#)

- 커밋 단위가 파일이 아니라 체인지셋이라는 점입니다. CVS에서라면 여러 개의 파일을 한꺼번에 커밋하더라도 각각의 파일마다. 리비전이 따로 붙습니다. 반면 Subversion에서는 파일별 리비전이 없고 한번 커밋 할 때마다 변경 사항별로 리비전이 하나씩 증가합니다.
- CVS에 비해 엄청나게 빠른 업데이트/브랜칭/태깅 시간.
- CVS와 거의 동일한 사용법. CVS 사용자라면 누구나 어려움 없이 금방 배울 수 있습니다.
- 파일 이름변경, 이동, 디렉토리 버전 관리도 지원.
- 원자적(atomic) 커밋. CVS에서는 여러 파일을 커밋하다가 어느 한 파일에서 커밋이 실패했을 경우 앞의 파일만 커밋이 적용되고 뒤의 파일들은 그대로 남아있게 됩니다. Subversion은 여러개의 파일을 커밋하더라도 커밋이 실패하면 모두 이전 상태로 되돌아 갑니다.
- 양방향 데이터 전송으로 네트워크 소통량(트래픽) 최소화.
- 트리별, 파일별 접근 제어 리스트. 저장소 쓰기 접근을 가진 개발자라도 아무 소스나 수정하지 못하게 조절 할 수 있습니다.
- 저장소/프로젝트별 환경 설정 가능
- 확장성을 염두에 둔 구조, 깔끔한 소스

2.2 설치 준비 작업 ¶ [\[edit\]](#)

이 장에서는 리눅스, 유닉스 등의 POSIX 호환 운영체제에서 소스를 컴파일하고 설치하는 방법을 다루고 있습니다.

Microsoft Windows에서의 사용은 [Microsoft Windows에서 사용하기](#) 장을 참조하십시오.

각각 리눅스 배포판 및 [FreeBSD](#), [NetBSD](#) 등의 전용 패키지에 대해서는 운영체제별 전용 패키지 장을 참조하십시오.

설치를 위해 준비해야 할 일.

2.3 사용 할 각각의 파일들 구하기 ¶ [\[edit\]](#)

- Subversion 소스파일 <http://subversion.tigris.org> subversion-1.2.X.tar.gz Subversion은 최신 버전을 받습니다.
- Apache2 <http://httpd.apache.org> httpd-2.0.XX.tar.gz Apache 2 도 최신 버전을 받습니다. 1.3버전은 연동이 불가능 합니다.
- Berkeley DB <http://www.sleepycat.com/download/index.shtml> db-4.3.28.NC.tar.gz Berkeley DB 는 버전을 꼭 4.3.28을 사용하여야 합니다. (*Subversion 버전 1.1 부터는 굳이 버클리 DB를 사용하지 않아도 됩니다. 파일 시스템을 사용할 수도 있습니다*)
- OpenSSL <http://www.openssl.org/source> openssl-0.9.7c.tar.gz OpenSSL은 [LASTEST](#)라고 된 것을 받습니다.

위의 파일들을 /root에 받습니다.

3 설치하기 ¶ [\[edit\]](#)

Subversion과 연관된 프로그램들을 컴파일 하고 설치하겠습니다.

3.1 OpenSSL 컴파일과 설치 ¶ [\[edit\]](#)

```
# tar vxzf openssl-0.9.7c.tar.gz
# cd openssl-0.97c
openssl-0.97c# ./config
openssl-0.97c# make
openssl-0.97c# make install
```

3.2 Berkeley DB 컴파일과 설치 ¶

[\[edit\]](#)

```
# tar vxzf db-4.3.28.NC.tar.gz
# cd db-4.3.28.NC
db-4.3.28.NC# cd build_unix
db-4.3.28.NC/build_unix# ../dist/configure
db-4.3.28.NC/build_unix# make
db-4.3.28.NC/build_unix# make install
db-4.3.28.NC/build_unix# echo "/usr/local/BerkeleyDB.4.3/lib" >> /etc/ld.so.conf
db-4.3.28.NC/build_unix# ldconfig
```

3.3 Apache 컴파일과 설치 ¶

[\[edit\]](#)

Apache는 설치해도 되고 안 해도 됩니다. 웹으로 저장소를 공개한다거나. <http://> 프로토콜을 이용해서 subversion을 이용하고 싶다면 설치하도록 합니다.

```
# tar vxzf httpd-2.0.54.tar.gz
httpd-2.0.54# ./configure --prefix=/usr/local/apache2 --enable-suexec W
--enable-so --with-suexec-caller=bin W
--enable-ssl --with-ssl=/usr/local/ssl --enable-cache W
--enable-ext-filter --with-z=/usr/include --enable-dav W
--with-dbm=db4 --with-berkeley-db=/usr/local/BerkeleyDB.4.2
httpd-2.0.54# make
httpd-2.0.54# make install
```

3.4 Subversion 컴파일과 설치 ¶

[\[edit\]](#)

데비안의 경우 zlib1g-dev, libxml2-dev, libexpat1-dev의 패키지가 필요합니다. 다른 배포판의 경우도 거의 같은 이름으로 된 패키지가 있을 것입니다. 이 패키지들은 라이브러리와 헤더 파일을 포함하고 있는 것들입니다.

앞에서 Apache를 설치했을 경우

```
# tar vxzf subversion-1.2.1.tar.gz
# cd subversion-1.2.1
subversion-1.2.1# ./configure --with-zlib --with-ssl=/usr/local/ssl W
--with-apr=/usr/local/apache2 W
--with-apr-util=/usr/local/apache2 W
--with-apxs=/usr/local/apache2/bin/apxs W
--with-dbm=db4 --with-berkeley-db=/usr/local/BerkeleyDB.4.3
subversion-1.2.1# make
subversion-1.2.1# make install
```

Apache를 설치하지 않았을 경우

```
# tar vxzf subversion-1.2.1.tar.gz
# cd subversion-1.2.1
subversion-1.2.1# ./configure --with-zlib --with-ssl=/usr/local/ssl W
--with-dbm=db4 --with-berkeley-db=/usr/local/BerkeleyDB.4.3
subversion-1.2.1# make
subversion-1.2.1# make install
```

4 세부 설정 ¶

[\[edit\]](#)

4.1 저장소 만들기 ¶

[\[edit\]](#)

소스를 저장할 공간을 만들어야 합니다. 저장소(Repository)는 프로젝트 마다 하나씩 있어야 합니다. 저장소 안

에 프로젝트의 소스가 다 들어가게 되며 다른 프로젝트를 진행한다면 그 프로젝트를 위한 저장소를 하나 더 만들어 주어야 합니다. /home/svn안에 저장소를 만들도록 하겠습니다. 앞으로 저장소를 추가 할 때에는 /home/svn 아래에 추가하면 됩니다. 꼭 /home/svn에 하지 않아도 되며 다른 곳에 해도 됩니다.

버클리DB를 이용한 저장소 만들기

```
# mkdir /home/svn
# cd /home/svn/
/home/svn# svnadmin create --fs-type bdb sample
```

파일시스템을 이용한 저장소 만들기

```
# mkdir /home/svn
# cd /home/svn/
/home/svn# svnadmin create --fs-type fsfs sample
```

svnadmin으로 sample이라는 저장소를 만들면 /home/svn 아래 sample이라는 디렉토리가 생깁니다. 이 디렉토리 안에는 Subversion이 제어하는 파일들이 들어있습니다. 이 디렉토리 안의 파일들은 Berkeley DB 형식으로 되어 있습니다. DB 파일들은 수정하는 일이 없도록 합니다. sample 저장소의 전체 경로는 /home/svn/sample 이 됩니다.

이제부터 sample 저장소를 계속 사용하여 설명을 하겠습니다.

4.1.1 공동 작업을 위한 저장소 그룹 설정 ¶

[\[edit\]](#)

svn+ ssh://로 작업을 하려면 시스템 계정을 만들어야 합니다. 대부분 계정을 만들고 그룹을 하나로 묶는데, 이럴 경우 그룹에 소속된 사용자들에게도 저장소 쓰기 권한을 주어야 합니다. 그래서 저장소의 그룹 권한을 조정해 주어야 합니다. 그룹 쓰기 권한을 주지 않으면 저장소를 만든 계정만 저장소에 접근이 가능하고, 그룹에 속해 있더라도 다른 사용자는 저장소에 접근 할 수 없게 됩니다.

```
# chmod -R g+w sample
```

4.2 Apache 설정 ¶

[\[edit\]](#)

Apache를 설치했다면 Apache 설정을 해주어야 합니다.

Apache가 저장소에 접근할 수 있도록 소유자와 권한을 바꿉니다. Apache는 기본적으로 설치하면 nobody와 nogroup로 실행됩니다.

```
# cd /home/svn
/home/svn# chown -R nobody.nogroup sample
```

Apache를 /usr/local/apache2에 설치했으므로 설정파일은 /usr/local/apach2/conf/httpd.conf 입니다. dav, dav_svn 모듈이 설정되어 있는지 확인 합니다. 주석처리 되어 있으면 주석을 없애고 없다면 아래 두줄을 추가합니다.

```
LoadModule dav_module      modules/mod_dav.so
LoadModule dav_svn_module   modules/mod_dav_svn.so
```

httpd.conf 파일 맨 뒷부분에 아래와 같이 추가 합니다. 설정을 저장한 뒤에 Apache를 재시작 합니다.

```
<Location /svn/sample>
  DAV svn
  SVNPath /home/svn/sample
</Location>
```

이렇게 설정을 하고 웹 브라우저에서 http://(Subversion과 Apache를 설치한 IP주소 또는 도메인)/svn/sample 로 접속을 합니다.

웹 브라우저에 아래와 같은 화면이 보이면 Subversion 저장소와 아파치가 잘 동작하고 있는 것입니다. 아래와 같이 나오지 않는다면 아파치 설정과 저장소의 소유자와 그룹을 다시 한 번 살펴보시기 바랍니다.

Revision 0: /

 Powered by Subversion version 1.0.0.

위와 같이 설정하면 누구든지(Anonymous) 웹 브라우저로 저장소를 볼 수 있고 Subversion 클라이언트를 이용해서 소스를 체크아웃, 익스포트, 커밋을 할 수 있습니다.

이렇게 실행 시켰으면 "# svn checkout http://(Subversion서버 IP또는 도메인)/svn/sample sample"을 입력합니다. "Checked out revision 0."이 나오면 제대로 설정이 된 것입니다. 아무나(Anonymous) 저장소에 접근해서 체크아웃, 커밋 등을 할 수 있습니다.

4.2.1 Apache에서 ID로 사용자 인증 ¶

[[edit](#)]

이제 ID를 통해 인증된 사용자만 소스를 체크아웃하고 커밋 할 수 있도록 설정 하겠습니다. 아파치에 사용할 패스워드 파일을 만듭니다. "# htpasswd -c 패스워드파일명 사용자ID"

```
# cd /usr/local/apache/conf
/usr/local/apache/conf# ../bin/htpasswd -c passwd sampleuser
New password:
Re-type new password:
```

"# htpasswd -c"는 패스워드 파일을 처음 만들 때의 옵션이고 사용자를 추가하고 싶을 때에는 "# htpasswd 패스워드파일명 사용자ID" 로 해주면 됩니다.

방금 수정했던 /usr/local/apache2/conf/httpd.conf 파일의 맨 마지막 부분에 추가한 부분을 다시 설정 합니다.

```
<Location /svn/sample>
  DAV svn
  SVNPath /home/svn/sample
  AuthType Basic
  AuthName "pyrasis's Repository"
  AuthUserFile /usr/local/apache2/conf/passwd
  Require valid-user
</Location>
```

4줄이 추가 되었습니다. AuthType Basic은 Apache 기본 패스워드 인증입니다. AuthName은 패스워드가 걸린 웹페이지에 뜨는 로그인창에 나올 문장입니다. AuthUserFile은 방금 전 만들었던 아파치 패스워드 파일입니다. 절대경로로 적어주어야 합니다. Require valid-user는 인증된 사용자만 볼 수 있게 한다는 것입니다.

이제 웹 브라우저로 다시 sample저장소로 접속해 보면 ID와 패스워드를 묻는 창이 나올 것입니다. 만든 ID와 패스워드를 입력하면 저장소를 볼 수 있습니다. 이렇게 되면 체크아웃, 커밋등을 할 때 ID와 암호를 물어보게 됩니다.

웹 브라우저로 저장소를 보는 것과 체크아웃은 아무에게나(Anonymous) 할 수 있게 하고 커밋은 지정된 사용자만 할 수 있도록 하려면 httpd.conf에서 설정한 부분을 아래와 같이 바꾸어 주면 됩니다.

```
<Location /svn/sample>
  DAV svn
  SVNPath /home/svn/sample
  AuthType Basic
  AuthName "pyrasis's Repository"
  AuthUserFile /usr/local/apache2/conf/passwd
  <LimitExcept GET PROPFIND OPTIONS REPORT>
    Require valid-user
  </LimitExcept>
</Location>
```

이렇게 하면 저장소를 보거나 체크아웃을 할 때는 ID와 패스워드를 묻지 않고 커밋이나 디렉토리, 파일복사 등의 저장소를 변경하는 작업을 할 때에는 ID와 패스워드를 물어보게 됩니다.

"# svn checkout http://(Subversion서버 IP또는 도메인)/svn/sample sample" 을 입력하면 ID와 패스워드를 물어웁니다. ID와 패스워드를 입력하고 나서 "Checked out revision 0." 이 출력되면 제대로 설정 된 것입니다.

4.3 svnserve를 사용한 서버 ¶

[[edit](#)]

Subversion의 고유 프로토콜인 svn://을 이용할 수 있는 svnserve를 사용해서 서버 설정을 해보겠습니다. 이것

을 사용하면 Apache를 사용한 것보다 체크아웃, 커밋 속도가 더 빠릅니다.

svnserve로 서버를 실행 시키면 3690번의 포트가 열립니다. sample 저장소가 /home/svn 아래에 있을 경우

```
# svnserve -d -r /home/svn/
```

이렇게 실행 시켰으면 "# svn checkout svn:///((Subversion서버 IP또는 도메인)/sample sample)"을 입력합니다. "Checked out revision 0."이 나오면 제대로 설정이 된 것입니다. 이것 또한 아무나(Anonymous) 저장소에 접근해서 체크아웃, 커밋 등을 할 수 있습니다.

4.3.1 svnserve에서 ID로 사용자 인증 ¶

[[edit](#)]

Subversion 0.33.0버전 이후부터 svnserve로 ID로 사용자 인증이 가능하게 되었습니다. 그 이전 버전에서 svnadmin으로 저장소를 만들면 저장소 디렉토리 아래에 conf 디렉토리가 생기지 않지만 0.33.0 버전이후에 svnadmin으로 저장소를 만들었다면 저장소 디렉토리 아래에 conf 디렉토리가 자동으로 생성됩니다. 이전 버전에서 먼저 저장소를 만들어 두었다면 저장소 디렉토리 /home/svn/sample 아래 conf 디렉토리를 만들어 줍니다. (/home/svn/sample/conf)

이제 각 저장소 디렉토리 아래 conf 디렉토리가 있습니다. /home/svn/sample/conf/svnserve.conf 파일이 svnserve의 설정 파일입니다. 0.33.0 버전 이전 만든 저장소에는 conf 디렉토리 및 svnserve.conf 파일이 없습니다. 그럴 경우에는 conf 디렉토리와 svnserve.conf 파일을 만들어 주면 됩니다.

svnserve.conf 파일을 아래와 같이 설정 합니다.

```
### This file controls the configuration of the svnserve daemon, if you
### use it to allow access to this repository. (If you only allow
### access through http: and/or file: URLs, then this file is
### irrelevant.)
```

```
### Visit http://subversion.tigris.org/ for more information.
```

```
[general]
```

```
### These options control access to the repository for unauthenticated
### and authenticated users. Valid values are "write", "read",
### and "none". The sample settings below are the defaults.
```

```
anon-access = none
```

```
auth-access = write
```

```
### The password-db option controls the location of the password
```

```
### database file. Unless you specify a path starting with a /,
```

```
### the file's location is relative to the conf directory.
```

```
### The format of the password database is similar to this file.
```

```
### It contains one section labelled [users]. The name and
```

```
### password for each user follow, one account per line. The
```

```
### format is
```

```
### USERNAME = PASSWORD
```

```
### Please note that both the user name and password are case
```

```
### sensitive. There is no default for the password file.
```

```
password-db = passwd
```

```
### This option specifies the authentication realm of the repository.
```

```
### If two repositories have the same authentication realm, they should
```

```
### have the same password database, and vice versa. The default realm
```

```
### is repository's uuid.
```

```
realm = pyrasis's Repository
```

anon-access = none으로 아무에게나(Anonymous) 저장소에 접근하는 것을 막았습니다. read로 하면 읽기만 가능하며 write로 해주면 읽고 쓰기가 가능해집니다.

auth-access = write는 ID로 인증된 사용자에게 쓰기 권한을 주는 것입니다.

password-db = passwd 이 설정은 svnserve의 패스워드 파일입니다 이전의 Apache 패스워드 파일과는 별개입니다. 아래 내용으로 /home/svn/sample/conf 아래 passwd 라는 이름으로 만듭니다. ID = 패스워드 형식입니다. 아직 암호화된 패스워드는 지원하지 않는 것 같습니다. 버전 업을 통해 개선 될 것 같습니다.

```
passwd
```

```
[users]
```

```
sampleuser = 02030104
```


이제 "# svn checkout svn:///((Subversion 서버의 IP또는 도메인))/sample sample" 을 해보면 ID와 패스워드를 입력하라고 나옵니다. 위에서 설정했던 ID와 패스워드를 입력하면 체크아웃이 되며 "Checked out revision 0." 이 나오면 설정이 제대로 된 것입니다.

4.4 SSH + svnserve 서버 ¶

[\[edit\]](#)

SSH2 데몬으로 svnserve를 터널링 하여 작동시킵니다. 이렇게 되면 svn+ssh:// 프로토콜을 사용하게 되며 사용자 인증은 시스템 계정으로 인증을 합니다. 구동시키는 방법은 따로 있지 않으며 SSH데몬만 떠 있으면 됩니다.

클라이언트에서 svn+ssh:// 를 사용하기

클라이언트에서 서버로 접속할 ID설정, 각 사용자 계정의 디렉토리에 .subversion이라는 디렉토리가 있습니다. 여기에 보면 config 라는 파일의 맨 마지막에 아래와 같이 추가합니다. ssh -l 서버에 접속할 ID

```
~/subversion/config
```

```
[tunnels]
```

```
ssh = ssh -l sampleuser
```

주의할 점은 IP주소나 도메인 뒤에 저장소의 절대 경로를 적어주어야 합니다. svnserve를 띄워서 /home/svn/과 같이 지정해 주지 않았기 때문에 각 저장소의 절대경로인 /home/svn/sample로 합니다.

```
# svn checkout svn+ssh:///((Subversion 서버의 IP주소 또는 도메인))/home/svn/sample sample
```

5 실제로 사용하기 ¶

[\[edit\]](#)

간단한 예제 프로그램을 통해서 Subversion의 사용법을 알아보도록 하겠습니다. 커맨드 라인 클라이언트를 통해 알아보겠습니다. X 윈도우, MS 윈도우용 GUI 클라이언트 프로그램에 대해서는 뒤에 설명하도록 하겠습니다.

5.1 에디터 설정 ¶

[\[edit\]](#)

Subversion에서 사용할 기본적인 에디터를 지정해야 합니다. 이것을 지정하지 않으면 커밋 등을 할 수 없게 됩니다.

각 사용자 계정의 디렉토리에는 .profile이나 .bash_profile 등의 환경 변수 등을 지정하는 파일이 있습니다. 여기에 아래와 같이 추가 합니다. 여기에서는 에디터를 vim으로 설정 하겠습니다. vim의 경로는 사용자마다 다를 수 있으므로 사용하고자 하는 시스템에 맞게 적어주십시오.

```
SVN_EDITOR=/usr/bin/vim
export SVN_EDITOR
```

5.2 기본 디렉토리 만들기 ¶

[\[edit\]](#)

앞서 설명 했던 trunk, branches, tags 디렉토리를 만들어 보겠습니다.

trunk 디렉토리의 생성, 앞에서 Apache를 통해 설정 했다면 http://를 svnserve로 설정했다면 svn://로 SSH를 사용한다면 svn+ssh://로 하여 서버 설정에 맞게 주소를 적어 주십시오. apache를 연동한 경우

```
# svn mkdir http:///((Subversion 서버의 IP주소 또는 도메인))/svn/sample/trunk
```

svnserve만 실행한 경우

```
# svn mkdir svn:///((Subversion서버 IP또는 도메인))/sample/trunk
```

위처럼 입력을 하면 vim이 실행되면서 아래와 같이 나올 것입니다. :q!로 빠져 나갑니다.

```
--This line, and those below, will be ignored--
```

```
A http:///((Subversion 서버의 IP주소 또는 도메인))/svn/sample/trunk
```

vim을 빠져 나왔다면 아래와 같이 나오는데(커밋 로그를 입력하지 않으면 아래와 이 나오게됩니다.) c를 누르고 엔터를 치면 디렉토리가 만들어 지게 됩니다.

Log message unchanged or not specified
a)bort, c)ontinue, e)dit

c를 입력한뒤 아래와 같이 나오면 디렉토리 생성은 성공한 것입니다. 이 방법대로 branches, tags 디렉토리도 만듭니다.

Committed revision 1.

만약 read-only에러가 나올 경우 conf/svnserve.conf에서 계정을 만들지 않아서 그렇습니다. anonymous로 사용할 경우 #general, #anon-access = read 가 주석 처리 되어 있는데 주석을 없애고 anon-access = write로 바꾸시면 됩니다.

제대로 만들어 졌는지 확인을 하려면 다음과 같이 입력 합니다. list는 저장소 안의 디렉토리들과 파일들을 표시해 줍니다.

```
# svn list http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample
branches/
tags/
trunk/
```

앞으로 디렉토리 생성, 삭제, 이름변경, 파일, 추가, 삭제, 복사, 이동과 커밋, 등을 할 때 vim이 실행되면서 어떤 것들이 바뀌는지 표시가 됩니다. 커밋 로그를 입력한뒤 vim을 종료하면 커밋이 완료 됩니다. 커밋 로그를 입력하지 않았을 경우 a)bort, c)ontinue, e)dit가 나오게 되는데 c)ontinue)로 계속 하면 됩니다. ⚠ 하지만! 커밋 로그 입력은 필수입니다. 소프트웨어 버전 관리에서 가장 중요한 것은 커밋 로그입니다. 어떤 코드를 어떻게 수정했고 디렉토리, 파일을 만들고 삭제 했는지를 꼼꼼히 기록해야합니다. 나중에 소스코드가 바뀌는 흐름을 따라가 고자 할때나 문제점(버그)을 추적할때 커밋 로그가 아주 유용하게 이용될 것입니다.

5.3 Import ¶

[[edit](#)]

맨 처음 프로젝트를 시작할 때 저장소에 소스들을 넣어야 합니다. 이럴 때 하는 것이 import 작업입니다. sampledir 이라는 디렉토리를 만든 뒤에 그 아래 다음과 같은 간단한 소스를 작성해 보십시오.

```
# mkdir sampledir
# cd sampledir
sampledir# vim sample.c

#include <stdio.h>

int main()
{
    printf("Sample Program Version 0.1\n");

    return 0;
}
```

이 소스를 저장소의 trunk 디렉토리에 import 하겠습니다. 아래 sampledir은 디렉토리입니다. 파일을 적으면 import되지 않습니다. 꼭 디렉토리를 만들고 그 디렉토리를 적어 주십시오. 저장소의 trunk 디렉토리에는 sampledir 디렉토리안의 sample.c 파일만 올라가게 되고 sampledir은 올라가지 않습니다. sampledir 아래 디렉토리를 만들었다면 그 디렉토리는 저장소의 trunk 디렉토리 아래에 올라가게 됩니다.

```
sampledir# cd ..
# svn import sampledir http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample/trunk
```

import도 위에서 디렉토리를 만들었을 때 처럼 vim이 실행되게 됩니다. import되는 파일들이 표시됩니다. :q!로 닫고 c를 입력하면 import 되게 됩니다.

import가 제대로 되었는지 확인해 봅시다. list 명령을 이용해 trunk 디렉토리에 무엇이 있나 보겠습니다.

```
# svn list http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample/trunk
sample.c
```

5.4 Checkout ¶

[[edit](#)]

이제 부터 Subversion을 이용해서 프로그램 소스를 관리 할 수 있습니다. checkout을 해서 어디서든 소스를 받아 볼 수 있습니다. 방금 import를 하기위해 만들었던 sampledir은 지워도 됩니다.

svn checkout은 svn co로 줄일 수 있습니다. "svn checkout 저장소주소 로컬디렉토리" 의 형식 입니다.

```
# svn checkout http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample/trunk sample
A sample/sample.c
Checked out revision 4.
```

checkout을 한 디렉토리 안에는 .svn 이라는 디렉토리가 있습니다. 이 디렉토리는 Subversion 저장소 정보가 들어 있기 때문에 지우면 안 됩니다.

5.5 Update ¶

[\[edit\]](#)

체크아웃 해서 받은 소스를 저장소의 최근 내용으로 업데이트 하는 명령입니다. 체크아웃 해서 받은 소스의 revision보다 저장소의 revision이 높으면 업데이트 하게 됩니다. 업데이트를 하게 되면 전부 다 받아오는 것이 아니라 변경 된 것들만 받아옵니다. 소스를 수정하기 전에 언제나 update 명령으로 소스를 가장 최신 revision으로 맞추고 작업을 해야 합니다.

svn update는 svn up로 줄여 사용할 수 있습니다.

```
sample# svn update
```

5.6 Commit ¶

[\[edit\]](#)

checkout 해서 받은 소스를 수정하고 저장소에 수정된 소스를 적용해 보겠습니다. 이 작업을 커밋(Commit)이라고 합니다.

체크아웃 받은 sample.c 소스를 아래처럼 수정합니다.

```
#include <stdio.h>

int main()
{
    printf("Sample Program Version 0.2\n");
    printf("Hello Subversion\n");

    return 0;
}
```

이제 커밋을 해 봅시다 커밋 명령은 체크아웃 해서 소스를 받은 디렉토리에서 해야 됩니다. svn commit은 svn ci로 줄여 쓸 수 있습니다. 커밋 명령을 내리면 커밋 로그를 작성하는 에디터가 실행 됩니다. 커밋 로그를 입력한 후 저장을 하면 커밋이 됩니다.

```
sample# svn commit
Sending      sample.c
Transmitting file data .
Committed revision 5.
```

5.7 Log ¶

[\[edit\]](#)

이제 저장소에 어떠한 것들이 변경 되었는지 확인 할 수 있는 log 명령에 대해 알아보겠습니다.

log 명령은 체크아웃 받은 디렉토리 안에서 해야 합니다. revision과 날짜, 누가 커밋을 했는지 알 수 있습니다. 여기서는 (no author)로 나옵니다. 이것은 서버 설정에서 아무나 커밋 할 수 있게 하여서 이렇게 나오는 것이고 ID를 통해 인증을 하도록 설정을 했으면 ID가 표시 됩니다.

```
sample# svn log
-----
r4 | (no author) | 2003-11-23 14:40:05 +0900 (Sun, 23 Nov 2003) | 1 line

-----
r1 | (no author) | 2003-11-23 14:39:00 +0900 (Sun, 23 Nov 2003) | 1 line

-----
```

--revision과 -r은 같습니다.

```
sample# svn log --revision 5:19  # revision 5부터 9까지의 로그를 출력합니다.
sample# svn log -r 19:5         # revision 19부터 5까지 역으로 출력합니다.
sample# svn log -r 8             # revision 8의 로그를 출력합니다.
```

하나의 파일이나 디렉토리 로그를 볼 수도 있습니다.

```
sample# svn log sample.c
sample# svn log http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample/trunk/sample.c
```

-v 옵션은 더 자세한 정보를 얻을 수 있습니다. 아래 M은 Modify(수정)의 표시 입니다. 소스 파일을 수정하고 커밋 했기 때문입니다.

```
sample# svn log -r 5 -v
```

```
-----
r5 | (no author) | 2003-11-23 14:42:34 +0900 (Sun, 23 Nov 2003) | 1 line
Changed paths:
  M /trunk/sample.c
-----
```

A 는 Add(추가) 표시 입니다. trunk라는 디렉토리를 만들었고 sample.c 파일을 import 했기 때문에 A(추가) 표시가 나오게 되는 것입니다.

```
sample# svn log -v
```

```
-----
r4 | (no author) | 2003-11-23 14:40:05 +0900 (Sun, 23 Nov 2003) | 1 line
Changed paths:
  A /trunk/sample.c
-----
```

```
-----
r1 | (no author) | 2003-11-23 14:39:00 +0900 (Sun, 23 Nov 2003) | 1 line
Changed paths:
  A /trunk
-----
```

5.8 Diff ¶

[\[edit\]](#)

diff 명령을 사용하면 예전 소스 파일과 지금의 소스 파일을 비교해 볼 수 있습니다. 위에서 나온 로그와 같이 우리는 revision 4(r4)에서 sample.c를 import 했습니다. 그리고 revision 5에서 소스를 수정하고 커밋 했습니다.

최초에 import 했던 소스 sample.c의 revision 4와 수정해서 커밋한 revision 5의 차이점을 diff 명령으로 출력해 보겠습니다. --revision 4만 적으면 현재 revision 5와 비교하게 됩니다.

```
sample# svn diff --revision 4 sample.c
Index: sample.c
```

```
=====
--- sample.c      (revision 4)
+++ sample.c      (working copy)
@@ -2,7 +2,8 @@

int main()
{
- printf("Sample Program Version 0.1Wn");
+ printf("Sample Program Version 0.2Wn");
+ printf("Hello SubversionWn");

return 0;
}
```

revision 4와 5를 비교 하고 싶으면 --revision 4:5 (-r 4:5)로 하면 됩니다. --revision 8:10 도 가능합니다.

```
sample# svn diff --revision 4:5 sample.c
Index: sample.c
```

```
=====
--- sample.c      (revision 4)
```

```
+++ sample.c (revision 5)
@@ -2,7 +2,8 @@

int main()
{
- printf("Sample Program Version 0.1Wn");
+ printf("Sample Program Version 0.2Wn");
+ printf("Hello SubversionWn");

return 0;
}
```

여기서 주의할 점은 CVS는 revision을 파일 하나하나에 다 매기지만 Subversion은 파일에 revision을 매기지 않고 소스 수정, 파일 복사, 이동, 삭제 등을 하고 그때 한 커밋으로 revision을 매깁니다. 그래서 다른 파일들이라도 같은 revision 번호를 가지게 됩니다.

5.9 Blame ¶

[\[edit\]](#)

Blame은 한 소스파일을 대상으로 각 리비전 대해서 어떤 행을 누가 수정했는지 알아보기 위한 명령입니다. 리비전, 커밋한 사용자의 ID, 소스 순서입니다.

```
sample# svn blame sample.c
4 sampleuser #include <stdio.h>
4 sampleuser
4 sampleuser int main()
4 sampleuser {
5 sampleuser printf("Sample Program Version 0.2Wn");
5 sampleuser printf("Hello SubversionWn");
4 sampleuser
4 sampleuser return 0;
4 sampleuser }
4 sampleuser
```

여기서는 커밋한 사용자가 한명 밖에 없으므로 전부 똑같이 나옵니다.

한 예로 여러사람이 커밋했을 경우 아래처럼 나옵니다.

```
4 sampleuser #include <stdio.h>
4 sampleuser
4 sampleuser int main()
4 sampleuser {
7 epifanes printf("Sample Program Version 0.3Wn");
6 pyrasis printf("Hello SubversionWn");
4 sampleuser
4 sampleuser return 0;
4 sampleuser }
4 sampleuser
```

특정 리비전만 지정해서 볼 수도 있습니다. 리비전을 지정하지 않으면 현재 리비전을 대상으로 합니다.

```
sample# svn blame -r 4 sample.c
```

5.10 lock ¶

[\[edit\]](#)

svn lock 명령으로 파일을 잠글 수 있습니다.

```
sample# svn lock hello.c
```

svn lock 명령으로 파일을 잠그었을 경우 왜 잠그었는지 로그를 기록할 수 있습니다. 파일을 잠그었을 경우 파일을 잠근 사용자만 수정을 해서 커밋을 할 수 있고 다른 사용자는 수정을 할 수 없습니다. 파일의 잠금을 해제할 경우에는 svn unlock 명령을 사용하면 됩니다. 파일 잠금기능은 여러명이 개발을 하고 있을 경우 한 파일에서 작업이 너무 많아 모두 끝내지 못하고 중간 중간에 커밋만 해놓았을 경우 다른 사람이 그 파일을 수정해버리면 하던 작업이 엉망이 되어 버립니다. 그래서 파일을 잠그어 작업이 끝날때 까지 한 사람만 커밋을 할 수 있도록 하는 것입니다. 작업이 끝나면 파일 잠금을 해제 하면 됩니다.

5.11 Add ¶

[\[edit\]](#)

svn add 명령으로 새 파일을 추가할 수 있습니다.

```
sample# svn add hello.c
A      hello.c
```

svn add로 파일을 추가한 뒤에는 svn commit으로 커밋을 해주어야 완전히 파일이 저장소에 추가됩니다.

```
sample# svn commit
```

물론 커밋 로그도 입력해야 됩니다.

5.12 Export ¶

[\[edit\]](#)

체크아웃은 Subversion이 버전관리를 할 수 있도록 각종 파일이 소스 폴더 안에 함께 생깁니다. 이것과는 달리 Export는 순수하게 프로그램의 소스만 가져올 수 있습니다. 만들어진 소스를 압축하여 릴리즈 할 때 사용합니다.

```
sample# svn export http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample2/trunk sample
```

--revision (-r)으로 revision을 지정하면 지정한 revision의 소스를 받아옵니다. 지정하지 않으면 가장 최근의 revision의 것을 가져옵니다.

5.13 Branch와 Tag ¶

[\[edit\]](#)

CVS에서의 Branch와 Tag는 Branch와 Tag를 위한 명령이 있습니다. CVS에서 Branch와 Tag를 하게 되면 저장소의 파일에는 Branch 또는 Tag 되었다는 표시가 함께 붙게 됩니다. 체크아웃 할 때에도 Branch와 Tag의 소스를 받아오려면 Branch, Tag를 지정하는 옵션을 주어야 했습니다.

CVS와는 달리 Subversion은 Branch와 Tag가 특별한 명령이 있는 것도 아니고 이런 것들을 한다고 해도 저장소에 특별히 Branch, Tag라고 기록이 남는 것도 아닙니다. Subversion에서 Branch와 Tag는 단순한 디렉토리 복사 명령으로 할 수 있고 Branch, Tag를 했더라도 디렉토리 복사와 같습니다.

5.13.1 Branch ¶

[\[edit\]](#)

Branch를 해야 할 경우는 앞서 설명 했듯이 프로젝트중 작은 분류로 나누어 개발 하거나 소스를 따로 분리하여 실험적인 코드를 작성할 경우 등 입니다.

Subversion에서는 Branch를 copy 명령으로 수행 합니다. trunk의 내용을 Branches안에 새로운 이름으로 복사하는 것입니다. 체크아웃 받은 소스에서 바로 copy를 할 수도 있고 원격에서 URL로 복사를 할 수도 있습니다. 아래 체크아웃 받은 것은 trunk만 받은 것이 아니고 sample 디렉토리 아래를 전부 받는 것입니다.

```
svn checkout http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample sample
sample# svn copy trunk branches/sample-branch
sample# svn commit
```

원격에서 URL로 copy를 하면 commit도 같이 이루어집니다. 체크아웃 받은 소스에서는 update를 해주어야 합니다.

```
# svn copy http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample/trunk W
http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample/branches/sample-branch
```

Branch된 소스를 받기 위해서는 branches/sample-branch를 체크아웃 하면 됩니다. trunk와 branche는 따로 revision을 가지지 않습니다. Subversion의 revision은 저장소 전체의 revision입니다.

```
# svn checkout W
http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample/branches/sample-branch W
sample-branch
```

5.13.1.1 Merge ¶

[\[edit\]](#)

위와 같이 우리는 trunk를 sample-branch로 Branch 했습니다. sample-branch를 수정하다가 trunk에도 반영하고 싶다면, merge 명령을 사용하면 됩니다. 하나하나 소스 코드를 옮겨서 입력하지 않아도 됩니다. 다만 merge 한 뒤에는 꼭 사람이 확인을 해 봐야겠죠.

sample-branch를 체크아웃 합니다.

```
# svn checkout W
```


[http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample/branches/sample-branch](http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample/branches/sample-branch) W
sample-branch

sample-branch의 sample.c를 다음과 같이 수정 합니다. printf("Hello WorldWn");를 추가 했습니다. 그리고 commit을 합니다. 지금 수정된 것은 revision 7입니다.

```
#include <stdio.h>
```

```
int main()
{
    printf("Sample Program Version 0.2Wn");
    printf("Hello SubversionWn");

    printf("Hello WorldWn");

    return 0;
}
```

이제 sample의 trunk를 체크아웃 합니다.

```
# svn checkout http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample/trunk sample
```

trunk의 sample.c는 아래와 같습니다.

```
#include <stdio.h>
```

```
int main()
{
    printf("Sample Program Version 0.2Wn");
    printf("Hello SubversionWn");

    return 0;
}
```

이제 sample-branch의 수정된 것을 trunk에 merge 해 보겠습니다. "svn merge -r N:N 저장소주소 체크아웃 된디렉토리" 형식 입니다. 아래는 저장소 주소 하나만 입력되어 있습니다. 이렇게 되면 지금 체크아웃한 소스와 merge를 하게 됩니다. merge할 revision 번호를 주의해 주십시오. 6:7은 r 6과 r 7의 차이점을 뜻합니다. sample-branch의 r 6과 r 7의 차이점을 지금의 체크아웃된 trunk에 적용하라는 것 입니다.

```
sample# svn merge -r 6:7 W
http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample/branches/sample-branch
U sample.c
sample# svn commit
sample# svn update
```

이제 sample.c를 열어보면 아래와 같이 sample-branch에서 수정한 것이 merge가 되어 있습니다.

```
#include <stdio.h>
```

```
int main()
{
    printf("Sample Program Version 0.2Wn");
    printf("Hello SubversionWn");

    printf("Hello WorldWn");

    return 0;
}
```

파일 하나만 merge를 할 수도 있습니다.

```
# svn merge -r 6:7 sample.c
```

저장소 주소끼리 merge를 할 수도 있습니다.

```
# svn merge http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample2/trunk W
http://\(Subversion 서버의 IP주소 또는 도메인\)/svn/sample2/branches/sample-branch
```

5.13.2 Tag ¶

[\[edit\]](#)

Tag는 만든 프로그램을 웹 사이트 등에 공개할 때 사용합니다. Tag도 Subversion에서는 Branch와 마찬가지로 디렉토리 복사(copy)와 같습니다. tags 디렉토리 안에는 일반적으로 릴리즈(발표)하는 버전별 디렉토리를 만들어 사용합니다.

0.1 버전을 발표할 때 0.1 버전의 순간을 tags 디렉토리에 복사하는 것입니다. 0.2가 되었을 때 tags아래 0.2 디렉토리로 복사합니다. 이렇게 되면 각각의 버전별로 소스를 관리 할 수 있습니다. 저장소에서는 실제로 복사가 되는 것은 아니고 변경된 점만 복사하기 때문에 저장소의 용량이 소스코드의 크기만큼 배로 늘어나지는 않습니다.

trunk의 소스를 0.1 버전으로 Tag, Branch와 마찬가지로 체크아웃 받은 소스에서도 할 수 있고 원격에서 URL로도 할 수 있습니다. 아래 체크아웃 받은 것은 trunk만 받은 것이 아니고 sample 디렉토리 아래를 전부 받는 것입니다.

```
# svn checkout http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample sample
sample# svn copy trunk tags/0.1
sample# svn commit
```

원격에서 URL로 복사합니다. 이 경우 commit도 같이 이루어집니다. 체크아웃 받은 소스는 update를 해주어야 합니다.

```
# svn copy http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample/trunk W
http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample/tags/0.1
```

이제 0.1로 Tag한 소스를 Export로 받아서 압축한 뒤에 릴리즈(공개)를 하면 됩니다.

```
# svn export http://(Subversion 서버의 IP주소 또는 도메인)/svn/sample/tags/0.1 sample-0.1
```

5.14 Revert ¶

[\[edit\]](#)

소스를 수정하면서 merge를 하다 보면 분명히 잘못 했을 경우가 생깁니다. 그럴 때 하는 것이 revert입니다. revert는 단어 뜻 그대로 되돌리는 명령입니다. 커밋을 하기 전에만 되돌릴 수 있습니다. 커밋 하기전의 체크아웃 받은 소스를 되돌리는 명령입니다. 원격 저장소의 것은 되돌릴 수 없습니다.

```
sample# svn revert sample.c
```

5.15 백업 및 복구 ¶

[\[edit\]](#)

저장소는 가장 중요한 공간이기 때문에 백업은 필수입니다. 저장소 디렉토리를 그대로 보관할 수도 있지만 백업과 복구 명령을 사용하는것이 편리합니다. Windows, 리눅스, BSD 등 운영체제에 관계없이 백업 및 복구가 가능합니다. Windows에서 백업한것을 리눅스에서 사용할 수도 있고 BSD에서 백업한 것을 Windows에서 사용할 수도 있습니다. 저장소의 서버를 옮길때에는 저장소 디렉토리를 옮기는 것이 아니라 저장소 백업을 한뒤 그 백업과 일을 이용하여 새 서버에서 복구를 하는 방식으로 옮겨야합니다.

5.15.1 Dump ¶

[\[edit\]](#)

sample 저장소를 백업합니다. 표준 입출력을 통해서 저장소의 내용을 파일로 생성합니다. svnadmin dump 명령을 사용하며 이 명령은 저장소 디렉토리 바깥에서 사용해야 합니다.

```
repos# ls
sample
repos# svnadmin dump sample > sample.dump
```

5.15.2 Load ¶

[\[edit\]](#)

저장소 백업 파일을 이용해서 저장소를 복구합니다. svnadmin load 명령을 사용합니다. 빈 저장소를 생성한 뒤 백업 파일을 이용해서 복구를 합니다.

```
repos# svnadmin create sample
repos# ls
sample  sample.dump
repos# svnadmin load sample < sample.dump
```

6 Microsoft Windows에서 사용하기 ¶

[\[edit\]](#)

Microsoft Windows에서도 Subversion을 사용할 수 있습니다. 소스를 컴파일하지 않고 설치 파일을 통해 간단하게 설치해서 사용할 수 있습니다. Windows에서도 리눅스, 유닉스와 똑같은 기능을 사용할 수 있습니다.

6.1 설치 파일 구하기 ¶

[\[edit\]](#)

Subversion Windows 설치파일

Subversion Windows <http://subversion.tigris.org/servlets/ProjectDocumentList?folderID=91>

Windows 용 설치 파일을 받습니다. ZIP으로 압축된 바이너리를 사용해도 상관없습니다.

svn-1.0.0-setup.exe

6.2 설치 ¶

[\[edit\]](#)

설치 파일을 받았다면 일반적인 Windows 프로그램을 설치하듯이 설치하면 됩니다. ZIP으로 압축된 것은 적당한 디렉토리에 압축을 해제한뒤 사용하면 됩니다.

6.3 사용하기 ¶

[\[edit\]](#)

지금 설치한것들은 Subversion 커맨드 라인 클라이언트와 저장소를 네트워크에서도 사용할 수 있도록 하는 서버 프로그램 들입니다. 커맨드라인 사용법은 리눅스, 유닉스와 똑같습니다. 다만 Windows에서는 명령 프롬프트(cmd.exe)에서 사용합니다.

ZIP으로 된 바이너리를 사용하려 한다면 명령 프롬프트에서 Subversion의 명령을 실행하기 위해 환경 변수의 시스템 변수 PATH에 Subversion 압축을 해제한 디렉토리를 추가합니다. 설치 파일로 설치했다면 자동으로 환경 변수에 추가됩니다.

커밋 로그를 입력할 수 있도록 환경 변수의 Administrator에 대한 사용자 변수에 변수이름 SVN_EDITOR, 값 notepad를 설정합니다. notepad가 아닌 다른 편집기를 이용하려면 편집기의 실행파일의 경로를 지정해 주면 됩니다.

저장소 만들기. C:\W에 repos라는 폴더를 만들었습니다. 명령 프롬프트를 실행합니다.

```
C:\WDocuments and Settings\WAdministrator>cd c:\Wrepos
```

버클리 DB를 이용한 저장소

```
C:\Wrepos>svnadmin create --fs-type bdb sample
```

파일시스템을 이용한 저장소

```
C:\Wrepos>svnadmin create --fs-type fsfs sample
```

체크아웃. svn://, http://를 이용한 체크아웃 방식은 위에서 설명한 방법과 똑같습니다.

윈도우 파티션에 있는 저장소에 직접 접근하는 방법.

```
C:\Wtemp>svn checkout file:///C:/repos/sample
```

svnserve를 사용한 서버

```
C:\W>svnserve -d -r C:\Wrepos
```

명령행에서 일일이 실행하는 불편함을 덜어주는 [SVNSERVE Manager](#)같은 프로그램을 이용할 수도 있습니다. svnserve의 동작/정지 상태를 트레이 아이콘으로 표시해 주며 시스템 시작시 svnserve를 자동으로 실행 하게 할 수 있습니다.

Windows용 Subversion 명령도 리눅스, 유닉스에서의 명령과 똑같습니다. 하지만 Windows에서는 그래픽 클라이언트가 있기 때문에 명령 프롬프트를 사용하는 일은 많지 않습니다. [TortoiseSVN](#)을 사용하면 팝업 메뉴를 이용해서 저장소 만들기, 체크아웃, 커밋 등 매우 편리하게 사용할 수 있습니다.

7 운영체제별 전용 패키지 ¶

[\[edit\]](#)

리눅스 배포판별(Redhat, Debian, [SuSE](#)) 전용 바이너리 패키지, [FreeBSD](#) 포트 컬렉션, [NetBSD](#) pkgsrc, Mac OS X 패키지 등 편리하게 설치할 수 있도록 운영체제별 전용 패키지가 제공되고 있습니다. 이것들을 사용하면 소스를 컴파일 하지 않고 바로 설치해서 사용할 수 있는 장점이 있습니다.

운영체제별 패키지는 아래 링크에서 받을 수 있습니다. http://subversion.tigris.org/project_packages.html

8 GUI 클라이언트 프로그램 ¶

[\[edit\]](#)

Subversion에서 기본적으로 지원하는 커맨드 라인 명령 svn은 사용하기에 불편한 점이 많습니다. 앞으로 소개 할 것들은 MS Windows, X Window 등에서 사용 가능한 Subversion 클라이언트 프로그램 입니다.

8.1 TortoiseSVN ¶

[\[edit\]](#)

MS Windows용 GUI 클라이언트 프로그램입니다. CVS GUI 클라이언트 프로그램으로 유명한 [TortoiseCVS](#)와 거의 같은 인터페이스를 가지고 있습니다. <http://tortoisesvn.tigris.org>

8.2 Ankhsvn ¶

[\[edit\]](#)

Visual Studio .NET 애드인 형식의 Subversion 클라이언트 프로그램입니다. VS.NET과 통합성이 매우 높습니다. VS.NET의 솔루션 뷰에서 커밋, 업데이트 등의 작업이 가능하며 솔루션 뷰의 각 파일에 수정되었거나 수정 되지 않은 파일의 상태를 표시해줍니다. <http://ankhsvn.tigris.org>

8.3 RapidSVN ¶

[\[edit\]](#)

크로스 플랫폼 Subversion 클라이언트 프로그램입니다. Windows, 리눅스, BSD의 X Window에서 사용할 수 있습니다. <http://rapidsvn.tigris.org>

9 웹 인터페이스 ¶

[\[edit\]](#)

저장소를 웹브라우저로 편하게 볼 수 있는 인터페이스들입니다.

9.1 ViewCVS ¶

[\[edit\]](#)

CVS 웹 인터페이스로 유명합니다. 아파치와 mod_python 기반으로 동작하며 Subversion 파이썬 바인딩으로 만들어져 있습니다. 최신버전은 Subversion도 지원하고 있습니다. 유닉스, 리눅스, Windows 모두 사용할 수 있습니다. <http://sourceforge.net/projects/viewcvs>

[윈도우에서 Subversion과 ViewCVS 사용하기](#)

9.2 WebSVN ¶

[\[edit\]](#)

Subversion 전용 웹 인터페이스입니다. Subversion svnlook과 연동하여 웹으로 표시합니다. 아파치와 php가 필요합니다. <http://websvn.tigris.org>

10 쓰다가 발생하는 사소한 문제 ¶

[\[edit\]](#)

10.1 svn: Entry 'file name' has unexpectedly changed special status ¶

[\[edit\]](#)

나의 경우 symbolic link파일을 svn add하였다가 그냥 지우고 새로운 파일을 만들어 svn ci하자 위와 같은 문제가 발생했다. 이경우 다음과 같이 처리한다.

1. 우선 원래의 symbolic link를 다시 복구 한다.
2. 그다음 svn ci을 한다
3. 그 다음 svn rm symbolic link한다
4. 그 다음 다시 내가 바꾸고자 하는 파일을 만든다
5. svn add, svn ci한다.

예를 들면 다음과 같다.

```
$ ln -s /some/dir/file ./file1
$ svn add file1
$ rm file1
$ cat > file1
my
new
version
$ svn ci
svn: Entry 'file1' has unexpectedly changed special status
$ rm file1
$ ln -s /some/dir/file ./file1
$ svn ci
$ svn rm file1
$ cat > file1
my
new
version
$ svn add file1
$ svn ci
```

[Tags:](#)

- [subversion](#)
- [svn](#)
- [버전 관리](#)

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

W3C XHTML 1.0 W3C CSS JAVASCRIPT POWERED

Last modified: 2006-10-17 11:12:48, Processing time: 0.0635 sec

[무료 IE7 다운로드](#)

빠르고 안전한 브라우저. 지금 바로 Google에
최적화된 최신 IE7 다운로드

[Subversion Made Better](#)

Transform subversion into a multi site
development solution